

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Programação Concorrente
Exame de Época de Recurso, Verão de 2023/2024
Duração: 3 horas

1. [4] Considere a classe **UnsafePairSuccession**, que tem por objectivo representar uma sequência de elementos, consumidos de forma sequencial e em pares de elementos consecutivos, através do método **next**. Assuma que os elementos do *array* não são alterados após a construção da instância da sequência.

```
class UnsafePairSuccession<T>(  
    private val items: Array<T>  
) {  
    private var currIndex = 0  
    fun nextConsecutiveItemsPair(): Pair<T, T>? {  
        if (currIndex + 2 > items.size) return null  
        val first = items[currIndex++]  
        val second = items[currIndex++]  
        return Pair(first, second)  
    }  
}
```

- a. [1,5] Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.
b. [2,5] Realize uma versão *thread-safe* desta classe, sem recurso ao uso de *locks*.
2. [4] Realize a classe **ComputationFuture** com a interface apresentada em seguida.

```
class ComputationFuture<T>(val provider: () -> T) {  
    @Throws(ExecutionException::class, TimeoutException::class, InterruptedException::class)  
    fun get(timeout: Duration): T {...}  
}
```

Cada instância pode estar num de três estados: *running*, *failed*, *succeeded*. A criação de uma instância coloca-a no estado *running* e cria uma *thread* virtual para executar a função **provider**. A instância evolui para o estado *succeeded* quando a execução da função terminar com sucesso ou para o estado *failed*, caso essa execução resulte no lançamento de uma excepção.

A função **get** bloqueia a *thread* invocante enquanto o estado da instância for *running*, retornando o resultado da execução de **provider**, caso o estado final seja *succeeded*, ou lançando uma excepção do tipo **ExecutionException** com a excepção lançada pela função **provider**, caso o estado final seja *failed*.

A função **get** pode ainda terminar a espera caso o tempo definido pelo parâmetro **timeout** seja ultrapassado sem que o estado deixe de ser *running*, lançando a excepção **TimeoutException**. A função **get** deve também ser sensível a interrupções. Tenha em consideração que a mesma instância de **ComputationFuture** pode ser usada por múltiplas *threads*.

3. [4] Realize o sincronizador **WaitQueue** com a interface apresentada em seguida.

```
class WaitQueue {
    @Throws(InterruptedException::class)
    fun await(timeout: Duration): Boolean {...}
    fun signalOne(): Thread? {...}
}
```

A função **await** bloqueia a *thread* invocante até que exista uma invocação da função **signalOne** e esta *thread* seja a selecionada para receber essa sinalização ou até que o tempo definido no parâmetro **timeout** seja ultrapassado. A função deve retornar **false** caso a saída seja por *timeout* ou **true** em caso contrário. A função deve também ser sensível a interrupções da *thread* onde a chamada foi realizada.

A função **signalOne** sinaliza a *thread* que está há mais tempo em espera, retornando a referência para a *thread* que foi desbloqueada em consequência da sinalização, caso esta exista. Caso não existam *threads* em espera, **signalOne** não tem qualquer efeito e retorna **null**.

Valorizam-se as implementações em que **signalOne** procure acordar apenas uma das *threads* em **await**.

4. [4] Realize o sincronizador **CyclicCountDownLatch** com a interface apresentada em seguida.

```
class CyclicCountDownLatch(val initialCount: Int) {
    init { require(initialCount > 0) }

    suspend fun countDown(): Int { ... }
    suspend fun await(): Unit { ... }
}
```

A função **countDown** decrementa o contador. Se o contador chegar a zero, o contador deve ser repostado de imediato com **initialCount** e todas as corrotinas que estejam nesse momento em espera em **await** devem terminar a espera. O retorno de uma chamada à função **countDown** indica o número de corrotinas que terminaram a espera com sucesso devido a essa chamada.

A função **await** deve suspender a execução até que uma chamada à função **countDown** tenha decrementado o contador até ao valor zero e deve ser sensível ao cancelamento da corrotina onde é chamada.

5. [4] Realize a função

```
fun <T> toFuture(scope: CoroutineScope, provider: suspend () -> T)
    : CompletableFuture<T>
```

Esta função retorna imediatamente um *future*, que se deve completar com o resultado da avaliação da função **provider** no âmbito de uma nova corrotina. Note que a avaliação de **provider** pode resultar numa exceção.

Duração: 3 horas
ISEL, 5 de julho de 2024